BCA PRACTICAL EXAMINATION MANUAL

DIGITAL IMAGE PROCESSING

All 25 Practical Programs: Questions, Objectives, Shortest Memorizable Code & Expected Outputs

Exam Quick Reference: This manual contains all 25 Digital Image Processing practical programs crafted for maximum simplicity, correctness, and quick handwriting memorization. All programs run with image.jpg (and reference.jpg for Q14). Install dependencies via pip install -r requirements.txt.

Question 1: Image Loading and Grayscale Conversion

File: image_loading_grayscale.py

Objective: *Load and display a color image and grayscale image. Convert color to grayscale.*

```
# Question 1: Image Loading and Grayscale Conversion
import cv2
import matplotlib.pyplot as plt

color = cv2.imread('image.jpg')
rgb = cv2.cvtColor(color, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(color, cv2.COLOR_BGR2GRAY)

plt.subplot(1, 2, 1)
plt.imshow(rgb)
plt.title('Color')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(gray, cmap='gray')
plt.title('Grayscale')
plt.axis('off')

plt.show()

# Output: Displays original color and converted grayscale images
```

Expected Output: Displays original color and converted grayscale images

Question 2: Sampling and Quantization

File: sampling_quantization.py

Objective: *Reduce spatial resolution (sampling) and gray levels (quantization) on the common image.*

```
# Question 2: Sampling and Quantization
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
sampled = cv2.resize(cv2.resize(img, (32, 32)), (img.shape[1], img.shape[0]), interpolation=cv2.INTER_NEAREST)
quantized = (img // 64) * 64

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(sampled, cmap='gray'), plt.title('Sampled (32x32)'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(quantized, cmap='gray'), plt.title('Quantized (4 levels)'), plt.axis('off')
plt.show()

# Output: Original, downsampled, and 4-level quantized images
```

Expected Output: Original, downsampled, and 4-level quantized images

Question 3: Spatial and Intensity Resolution

File: spatial_intensity_resolution.py

Objective: *Demonstrate differences between spatial resolution (pixel grid) and intensity resolution (gray levels).*

```
# Question 3: Spatial and Intensity Resolution
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
low_spatial = cv2.resize(cv2.resize(img, (64, 64)), (img.shape[1], img.shape[0]), interpolation=cv2.INTER_NEARES
low_intensity = (img // 128) * 128

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original (8-bit)'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(low_spatial, cmap='gray'), plt.title('Low Spatial (64x64)'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(low_intensity, cmap='gray'), plt.title('Low Intensity (1-bit)'), plt.axis('off
plt.show()

# Output: Compares spatial resolution reduction with intensity resolution reduction
```

Expected Output: Compares spatial resolution reduction with intensity resolution reduction

Question 4: Adjacency and Connectivity

File: adjacency_connectivity.py

Objective: *Compute and demonstrate 4-neighbour and 8-neighbour connectivity on a binary array.*

```
# Question 4: Adjacency and Connectivity
import numpy as np
import cv2

# 5x5 binary image with diagonal 1s
arr = np.array([
    [1, 0, 0, 0, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1]], dtype=np.uint8)

n4, _ = cv2.connectedComponents(arr, connectivity=4)
n8, _ = cv2.connectedComponents(arr, connectivity=8)

print(f"4-Connected Components: {n4 - 1}")
print(f"8-Connected Components: {n8 - 1}")

# Output:
# 4-Connected Components: 5
# 8-Connected Components: 1
```

Expected Output:

Question 5: RGB, CMY, and HSI Color Models

File: rgb_cmy_hsi.py

Objective: *Separate color image into RGB components and convert to CMY and genuine HSI.*

```
# Question 5: RGB, CMY, and HSI Color Models
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) / 255.0
r, g, b = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

# CMY conversion
cmy = 1.0 - rgb

# HSI conversion
i = (r + g + b) / 3.0
num = 0.5 * ((r - g) + (r - b))
den = np.sqrt((r - g)**2 + (r - b) * (g - b)) + 1e-6
theta = np.arccos(np.clip(num / den, -1.0, 1.0))
h = np.where(b > g, 2 * np.pi - theta, theta) / (2 * np.pi)
s = 1.0 - (3.0 / (r + g + b + 1e-6)) * np.minimum(np.minimum(r, g), b)
hsi = np.stack([h, s, i], axis=-1)

plt.subplot(1, 3, 1), plt.imshow(rgb), plt.title('RGB'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(cmy), plt.title('CMY'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(hsi), plt.title('HSI'), plt.axis('off')
plt.show()

# Output: Original RGB, converted CMY, and true HSI color images
```

Expected Output: Original RGB, converted CMY, and true HSI color images

Question 6: Hue, Saturation, and Intensity Modification

File: hue_saturation_intensity.py

Objective: *Modify hue, saturation, and intensity/value channels and display the result.*

```
# Question 6: Hue, Saturation, and Intensity Modification
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV).astype(np.float32)

# Modify Hue (+30), Saturation (x1.4), Value/Brightness (x1.2)
hsv[:, :, 0] = (hsv[:, :, 0] + 30) % 180
hsv[:, :, 1] = np.clip(hsv[:, :, 1] * 1.4, 0, 255)
hsv[:, :, 2] = np.clip(hsv[:, :, 2] * 1.2, 0, 255)

modified = cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2RGB)
orig = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.subplot(1, 2, 1), plt.imshow(orig), plt.title('Original'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(modified), plt.title('Modified H-S-V'), plt.axis('off')
plt.show()

# Output: Original image and modified Hue, Saturation, and Value/Brightness image
```

Expected Output: Original image and modified Hue, Saturation, and Value/Brightness image

Question 7: Color Histograms

File: color_histograms.py

Objective: *Compute and plot the color histograms for Red, Green, and Blue channels.*

```
# Question 7: Color Histograms
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
colors = ('b', 'g', 'r')

for i, col in enumerate(colors):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=col, label=col.upper() + ' channel')

plt.title('Color Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')
plt.legend()
plt.show()

# Output: Plots the intensity histograms for Red, Green, and Blue channels
```

Expected Output: Plots the intensity histograms for Red, Green, and Blue channels

Question 8: Image Negative

File: image_negative.py

Objective: *Apply intensity inversion transformation s = 255 - r.*

```
# Question 8: Image Negative
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
negative = 255 - img

plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(negative, cmap='gray')
plt.title('Negative')
plt.axis('off')

plt.show()

# Output: Original and negative images
```

Expected Output: Original and negative images

Question 9: Log Transformation

File: log_transformation.py

Objective: *Apply logarithmic transformation s = c * log(1 + r) to enhance dark regions.*

```
# Question 9: Log Transformation
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
c = 255.0 / np.log(1.0 + float(np.max(img)))
log_img = np.uint8(c * np.log(1.0 + img.astype(np.float32)))

plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(log_img, cmap='gray'), plt.title('Log Transformed'), plt.axis('off')
plt.show()

# Output: Original and log-transformed contrast-enhanced image
```

Expected Output: Original and log-transformed contrast-enhanced image

---

Question 10: Gamma Transformation

File: gamma_transformation.py

Objective: *Apply power-law (gamma) transformation using two gamma values (0.5 and 2.0).*

```
# Question 10: Gamma Transformation
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
gamma_low = np.uint8(255 * (img / 255.0) ** 0.5)
gamma_high = np.uint8(255 * (img / 255.0) ** 2.0)

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(gamma_low, cmap='gray'), plt.title('Gamma = 0.5'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(gamma_high, cmap='gray'), plt.title('Gamma = 2.0'), plt.axis('off')
plt.show()

# Output: Original and power-law (gamma) transformed images with gamma=0.5 and 2.0
```

Expected Output: Original and power-law (gamma) transformed images with gamma=0.5 and 2.0

Question 11: Piecewise Linear Transformations

File: piecewise_linear_transformation.py

Objective: *Demonstrate contrast stretching and binary thresholding piecewise linear functions.*

```
# Question 11: Piecewise Linear Transformations
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
# Contrast stretching: map [50, 180] to [0, 255]
stretched = np.uint8(np.clip((img - 50) * (255.0 / (180 - 50)), 0, 255))
# Thresholding: binary piecewise linear
_, thresh = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(stretched, cmap='gray'), plt.title('Contrast Stretched'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(thresh, cmap='gray'), plt.title('Thresholded'), plt.axis('off')
plt.show()

# Output: Original, contrast-stretched, and binary thresholded images
```

Expected Output: Original, contrast-stretched, and binary thresholded images

---

Question 12: Histogram Equalization

File: histogram_equalization.py

Objective: *Apply histogram equalization to enhance global contrast of a grayscale image.*

```
# Question 12: Histogram Equalization
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
equalized = cv2.equalizeHist(img)

plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(equalized, cmap='gray'), plt.title('Equalized'), plt.axis('off')
plt.show()

# Output: Original and histogram equalized grayscale images
```

Expected Output: Original and histogram equalized grayscale images

---

Question 13: Histogram Stretching

File: histogram_stretching.py

Objective: *Perform min-max normalization/stretching to span the full dynamic range [0, 255].*

```
# Question 13: Histogram Stretching
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
stretched = np.uint8((img - img.min()) * (255.0 / (img.max() - img.min())))

plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(stretched, cmap='gray'), plt.title('Stretched'), plt.axis('off')
plt.show()

# Output: Original and min-max histogram stretched images
```

Expected Output: Original and min-max histogram stretched images

---

Question 14: Histogram Matching (Specification)

File: histogram_matching.py

Objective: *Match the histogram of image.jpg to reference.jpg using CDF lookup table mapping.*

```
# Question 14: Histogram Matching
import cv2
import numpy as np
import matplotlib.pyplot as plt

src = cv2.imread('image.jpg', 0)
ref = cv2.imread('reference.jpg', 0)

# Compute cumulative distribution functions (CDFs)
s_cdf = np.cumsum(np.histogram(src, 256, [0, 256])[0]) / src.size
r_cdf = np.cumsum(np.histogram(ref, 256, [0, 256])[0]) / ref.size

# Map source intensities to reference intensities
lut = np.array([np.argmin(np.abs(s_cdf[i] - r_cdf)) for i in range(256)], dtype=np.uint8)
matched = lut[src]

plt.subplot(1, 3, 1), plt.imshow(src, cmap='gray'), plt.title('Source'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(ref, cmap='gray'), plt.title('Reference'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(matched, cmap='gray'), plt.title('Matched'), plt.axis('off')
plt.show()

# Output: Source image matched to the histogram of reference image
```

Expected Output: Source image matched to the histogram of reference image

---

Question 15: Spatial Correlation and Convolution

File: correlation_convolution.py

Objective: *Perform spatial filtering with an asymmetric kernel, demonstrating correlation vs 180-degree flipped convolution.*

```
# Question 15: Spatial Correlation and Convolution
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
# Asymmetric kernel to distinguish correlation from convolution
kernel = np.array([[1, 2, 3], [0, 0, 0], [-1, -2, -3]], dtype=np.float32)

corr = cv2.filter2D(img, -1, kernel)              # Correlation (unflipped)
conv = cv2.filter2D(img, -1, np.flip(kernel))     # Convolution (180-degree flipped)

plt.subplot(1, 2, 1), plt.imshow(corr, cmap='gray'), plt.title('Correlation'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(conv, cmap='gray'), plt.title('Convolution'), plt.axis('off')
plt.show()

# Output: Demonstrates spatial correlation and convolution using flipped kernel
```

Expected Output: Demonstrates spatial correlation and convolution using flipped kernel

---

Question 16: Smoothing Filters

File: smoothing_filters.py

Objective: *Apply mean filter and Gaussian filter to reduce image noise and blur details.*

```
# Question 16: Smoothing Filters
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
mean_blur = cv2.blur(img, (5, 5))
gaussian_blur = cv2.GaussianBlur(img, (5, 5), 0)

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(mean_blur, cmap='gray'), plt.title('Mean Filter'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(gaussian_blur, cmap='gray'), plt.title('Gaussian Filter'), plt.axis('off')
plt.show()

# Output: Original, mean blurred, and Gaussian blurred smoothed images
```

Expected Output: Original, mean blurred, and Gaussian blurred smoothed images

---

Question 17: Laplacian Sharpening

File: laplacian_sharpening.py

Objective: *Enhance fine details and edges using the 2nd-derivative Laplacian operator.*

```
# Question 17: Laplacian Sharpening
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
lap = cv2.Laplacian(img, cv2.CV_64F)
sharpened = np.uint8(np.clip(img - lap, 0, 255))

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(np.abs(lap), cmap='gray'), plt.title('Laplacian'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(sharpened, cmap='gray'), plt.title('Sharpened'), plt.axis('off')
plt.show()

# Output: Original, Laplacian edge response, and Laplacian sharpened image
```

Expected Output: Original, Laplacian edge response, and Laplacian sharpened image

---

Question 18: Unsharp Masking and High-Boost Filtering

File: unsharp_highboost.py

Objective: *Compute unsharp mask (f - f_blur) and produce unsharp (k=1) and high-boost (k=2) sharpened images.*

```
# Question 18: Unsharp Masking and High-Boost Filtering
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
blur = cv2.GaussianBlur(img, (5, 5), 0)
mask = img.astype(np.float32) - blur.astype(np.float32)

unsharp = np.uint8(np.clip(img + 1.0 * mask, 0, 255))
highboost = np.uint8(np.clip(img + 2.0 * mask, 0, 255))

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(unsharp, cmap='gray'), plt.title('Unsharp (k=1)'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(highboost, cmap='gray'), plt.title('High-Boost (k=2)'), plt.axis('off')
plt.show()

# Output: Original, unsharp masked, and high-boost filtered images
```

Expected Output: Original, unsharp masked, and high-boost filtered images

---

Question 19: Sobel and Prewitt Edge Detection

File: sobel_prewitt.py

Objective: *Detect horizontal and vertical gradient edges using Sobel and Prewitt operators.*

```
# Question 19: Sobel and Prewitt Edge Detection
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)

# Sobel Operator
sx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = np.uint8(np.clip(np.hypot(sx, sy), 0, 255))

# Prewitt Operator
px_k = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float32)
py_k = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float32)
px = cv2.filter2D(img, cv2.CV_64F, px_k)
py = cv2.filter2D(img, cv2.CV_64F, py_k)
prewitt = np.uint8(np.clip(np.hypot(px, py), 0, 255))

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(sobel, cmap='gray'), plt.title('Sobel Edges'), plt.axis('off')
plt.subplot(1, 3, 3), plt.imshow(prewitt, cmap='gray'), plt.title('Prewitt Edges'), plt.axis('off')
plt.show()

# Output: Original image along with Sobel and Prewitt edge detections
```

Expected Output: Original image along with Sobel and Prewitt edge detections

---

Question 20: 2D Fourier Transform and Inverse DFT

File: fourier_transform.py

Objective: *Compute 2D FFT, visualize centered magnitude spectrum, and reconstruct image via IDFT.*

```
# Question 20: 2D Fourier Transform and Inverse DFT
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)

# Inverse DFT
inv_shift = np.fft.ifftshift(fshift)
reconstructed = np.abs(np.fft.ifft2(inv_shift))

plt.subplot(1, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 3, 2), plt.imshow(magnitude_spectrum, cmap='gray'), plt.title('Magnitude Spectrum'), plt.axis('of
plt.subplot(1, 3, 3), plt.imshow(reconstructed, cmap='gray'), plt.title('Reconstructed'), plt.axis('off')
plt.show()

# Output: Original image, centered magnitude spectrum, and reconstructed image via IDFT
```

Expected Output: Original image, centered magnitude spectrum, and reconstructed image via IDFT

Question 21: Frequency-Domain Filters

File: frequency_filters.py

Objective: *Implement ideal circular low-pass and high-pass filters in the Fourier frequency domain.*

```
# Question 21: Frequency-Domain Filters
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# Distance from center
y, x = np.ogrid[:rows, :cols]
mask_lp = np.uint8(np.sqrt((x - ccol)**2 + (y - crow)**2) <= 30)
mask_hp = 1 - mask_lp

fshift = np.fft.fftshift(np.fft.fft2(img))
low_pass = np.abs(np.fft.ifft2(np.fft.ifftshift(fshift * mask_lp)))
high_pass = np.abs(np.fft.ifft2(np.fft.ifftshift(fshift * mask_hp)))

plt.subplot(1, 2, 1), plt.imshow(low_pass, cmap='gray'), plt.title('Ideal Low Pass'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(high_pass, cmap='gray'), plt.title('Ideal High Pass'), plt.axis('off')
plt.show()

# Output: Low-pass filtered (smooth) and high-pass filtered (edges) frequency-domain images
```

Expected Output: Low-pass filtered (smooth) and high-pass filtered (edges) frequency-domain images

---

Question 22: DFT Properties (Translation & Scaling)

File: dft_properties.py

Objective: *Demonstrate Fourier translation invariance (magnitude unchanged) and spatial scaling property.*

```
# Question 22: DFT Properties
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)

# Translation property: spatial shift leaves magnitude spectrum invariant
shifted = np.roll(np.roll(img, 50, axis=0), 50, axis=1)

# Scaling property: spatial compression broadens frequency spectrum
scaled = cv2.resize(cv2.resize(img, (img.shape[1] // 2, img.shape[0] // 2)), (img.shape[1], img.shape[0]))

mag_orig = 20 * np.log(np.abs(np.fft.fftshift(np.fft.fft2(img))) + 1)
mag_shift = 20 * np.log(np.abs(np.fft.fftshift(np.fft.fft2(shifted))) + 1)
mag_scale = 20 * np.log(np.abs(np.fft.fftshift(np.fft.fft2(scaled))) + 1)

plt.subplot(2, 3, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(2, 3, 2), plt.imshow(shifted, cmap='gray'), plt.title('Translated'), plt.axis('off')
plt.subplot(2, 3, 3), plt.imshow(scaled, cmap='gray'), plt.title('Scaled (Compressed)'), plt.axis('off')
plt.subplot(2, 3, 4), plt.imshow(mag_orig, cmap='gray'), plt.title('Orig Spectrum'), plt.axis('off')
plt.subplot(2, 3, 5), plt.imshow(mag_shift, cmap='gray'), plt.title('Shift Spectrum'), plt.axis('off')
plt.subplot(2, 3, 6), plt.imshow(mag_scale, cmap='gray'), plt.title('Scaled Spectrum'), plt.axis('off')
plt.show()

# Output: Demonstrates DFT translation (magnitude invariance) and scaling properties
```

Expected Output: Demonstrates DFT translation (magnitude invariance) and scaling properties

Question 23: Morphological Operations

File: morphological_operations.py

Objective: *Apply basic morphological operations: Dilation, Erosion, Opening, and Closing.*

```
# Question 23: Morphological Operations
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
_, binary = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)
kernel = np.ones((5, 5), np.uint8)

erosion = cv2.erode(binary, kernel)
dilation = cv2.dilate(binary, kernel)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

plt.subplot(2, 3, 1), plt.imshow(binary, cmap='gray'), plt.title('Binary'), plt.axis('off')
plt.subplot(2, 3, 2), plt.imshow(erosion, cmap='gray'), plt.title('Erosion'), plt.axis('off')
plt.subplot(2, 3, 3), plt.imshow(dilation, cmap='gray'), plt.title('Dilation'), plt.axis('off')
plt.subplot(2, 3, 4), plt.imshow(opening, cmap='gray'), plt.title('Opening'), plt.axis('off')
plt.subplot(2, 3, 5), plt.imshow(closing, cmap='gray'), plt.title('Closing'), plt.axis('off')
plt.show()

# Output: Binary image with erosion, dilation, opening, and closing operations
```

Expected Output: Binary image with erosion, dilation, opening, and closing operations

Question 24: Hit-or-Miss Transformation

File: hit_or_miss.py

Objective: *Detect a specific isolated foreground pixel pattern using morphological hit-or-miss.*

```
# Question 24: Hit-or-Miss Transformation
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 10x10 binary image with a pattern
img = np.zeros((10, 10), dtype=np.uint8)
img[2:5, 2:5] = 255   # 3x3 block
img[7, 7] = 255       # isolated point

# Structuring element to find an isolated 1 surrounded by 0s
kernel = np.array([
    [-1, -1, -1],
    [-1,  1, -1],
    [-1, -1, -1]], dtype=np.int8)

output = cv2.morphologyEx(img, cv2.MORPH_HITMISS, kernel)

plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Binary Image'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(output, cmap='gray'), plt.title('Hit-or-Miss (Isolated Point)'), plt.axis('off
plt.show()

# Output: Detects an isolated single pixel using hit-or-miss morphological transformation
```

Expected Output: Detects an isolated single pixel using hit-or-miss morphological transformation

Question 25: Noise Addition and Removal

File: noise_removal.py

Objective: *Simulate 5 noise types (Gaussian, Salt & Pepper, Rayleigh, Gamma, Exponential) and filter with mean smoothing.*

```
# Question 25: Noise Addition and Removal
import cv2
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
img = cv2.imread('image.jpg', 0).astype(np.float32)

# Generate noise types
rand = np.random.rand(*img.shape)
sp = img.copy()
sp[rand < 0.02] = 0
sp[rand > 0.98] = 255

noises = {
    'Gaussian': img + np.random.normal(0, 25, img.shape),
    'Salt & Pepper': sp,
    'Rayleigh': img + np.random.rayleigh(20, img.shape),
    'Gamma': img + np.random.gamma(2, 10, img.shape),
    'Exponential': img + np.random.exponential(15, img.shape)
}

plt.figure(figsize=(10, 4))
for i, (name, noisy) in enumerate(noises.items()):
    noisy_img = np.uint8(np.clip(noisy, 0, 255))
    filtered = cv2.blur(noisy_img, (5, 5))
    plt.subplot(2, 5, i + 1), plt.imshow(noisy_img, cmap='gray'), plt.title(name), plt.axis('off')
    plt.subplot(2, 5, i + 6), plt.imshow(filtered, cmap='gray'), plt.title(name + ' Filtered'), plt.axis('off')

plt.tight_layout()
plt.show()

# Output: 5 noise types (Gaussian, Salt-and-Pepper, Rayleigh, Gamma, Exponential) and their mean-filtered result
```

Expected Output: 5 noise types (Gaussian, Salt-and-Pepper, Rayleigh, Gamma, Exponential) and their mean-filtered results